

Cyrus Scatter Viewport performance

Why a small proxy preview slows navigation,
and the implementation work that can fix it.

90.52 ms

Original median step

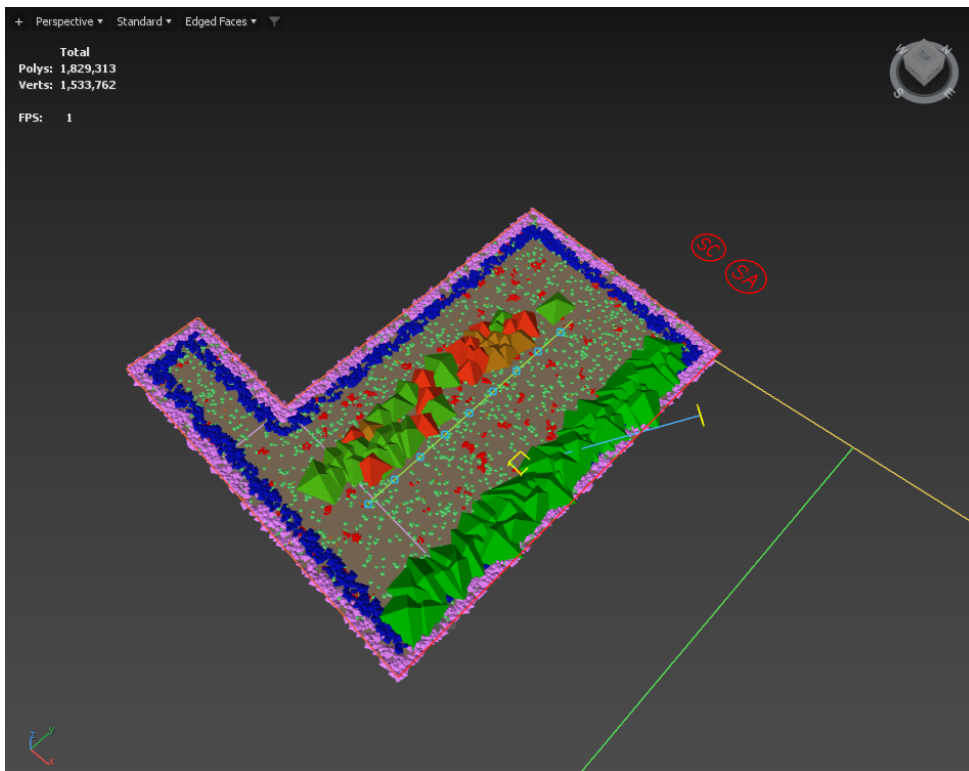
31.56 ms

Batched diagnostic prototype

0

Navigation rebuilds in main trials

Priority: batch per-instance proxy drawing, then remove repeated script work from redraw. Manual mode controls rebuilds; it still draws the existing preview.



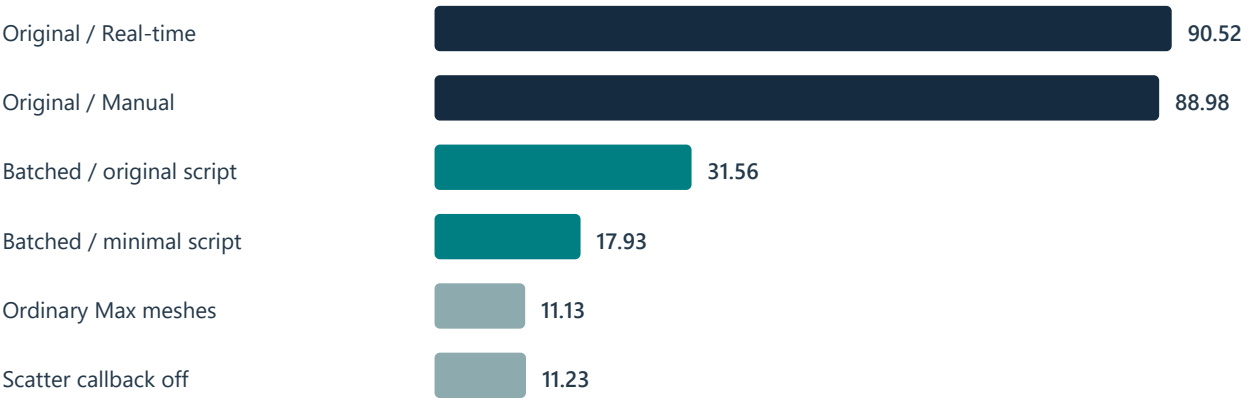
Supplied scene, original preview: 6,284 proxies / 37,704 triangles. The screenshot's idle FPS label is not a measurement. All headline timings are synchronous redraw/message-pump steps, not GPU-completed frames.

Guide to the findings

1. Executive findings	3
2. Reproduction and evidence	4
3. Root cause and code path	6
4. Experiments that establish the improvement path	7
5. Prioritized implementation guideline	9
6. Codebase-wide performance map	11
7. Validation and release gates	12
8. Reproduction tools and evidence register	13
9. Sources and related work	13

Median navigation-step wall time

Milliseconds; lower is better. Each row: 180 measured steps.



Batching and ordinary-mesh rows are disposable, matched-count experiments. They preserve displayed counts but do not certify identical shading or release behavior. The callback-off row is an isolation control. Analyzer drawing remains enabled in these rows.

Reading order: findings and code path for the diagnosis; experiments for the evidence; priorities and release gates for implementation. Raw recordings and investigation fixtures are retained in the repository.

1. Executive findings

The supplied Test Scene/SaveSelect 2.max was opened in the normal 3ds Max desktop application, tested through a disposable copy, and reopened unchanged at the end. The installed native modules and script were identified. Controlled camera movement, callback isolation, display budgets, per-layer timings, alternative drawing representations and actual mouse drags were tested.

The original preview contains **52,143 generated placements, 6,284 displayed pyramid proxies and 37,704 preview triangles**. This modest visible triangle count is expensive because the implementation opens and closes a native triangle batch for **each displayed instance**, every redraw. Cached placements do not make that drawing work free.

Three repeated trials of 60 measured navigation steps per condition found:

Condition	Median step	P95 step	Interpretation
Original, Real-time	90.52 ms	142.40 ms	Slow with valid, unchanged caches
Original, Manual	88.98 ms	135.66 ms	Manual does not solve drawing cost
Scatter drawing callback disabled	11.23 ms	19.15 ms	Scene and scatter controller remain present
Analyzer drawing callback disabled	87.50 ms	138.24 ms	Analyzer is a secondary contributor
Both Cyrus drawing callbacks disabled	6.14 ms	11.62 ms	Remaining scene is inexpensive on this path

Zero preview rebuilds, Analyzer runs or PFlow builds occurred during all 15 measured main-matrix trials. The result isolates steady drawing overhead from layout generation. Removing the scatter also removes this drawing burden, explaining why deletion appears to restore responsiveness immediately.

The strongest improvement experiment kept the original controller traversal, icon drawing, layer synchronization, cache checks and radius calls, but drew the proxy triangles in one world-space batch per layer. Median navigation step time fell to **31.56 ms**, approximately **65% lower** than the original 90.52 ms. This is an experimental result, not an installed product improvement.

Measurement boundary: a step includes a camera transform, redrawViews() and Windows message processing. It is not a GPU-fenced completed frame. Values below are wall times; their reciprocals must not be advertised as delivered viewport FPS. The on-screen idle FPS label is not a benchmark.

2. Reproduction and evidence

Host and provenance

Item	Observed value
Application	3ds Max 2027.1, 29.1.0.11426
Plugin	Cyrus Scatter 0.60; installed script matches repository script
Renderer	Corona 15 selected; no render or IR running
CPU	AMD Ryzen 5 5600X, 6 cores / 12 logical processors
GPU	NVIDIA GeForce RTX 3090; driver 32.0.16.1047
Physical memory	Approximately 95.9 GiB reported by Windows
Scene	50 scene objects, one Scatter controller, six enabled layers, one Analyzer
Saved viewport	Perspective, 953 x 750 pixels; two-view layout; Edged Faces
Nitrous setting	API value 3; VisualStyleMode reports Realistic; progressive rendering enabled
Test degradation control	AdaptiveDegradeNeverDegradeGeometry=true; original value false restored
CS Edit	No CS Edit modifier on this scene's Scatter controller

The visible viewport menu says Standard while the queried `VisualStyleMode` says Realistic; both observations are retained rather than silently normalized. The test does not certify the global adaptive-degradation switch or independently validate `viewport.GetFPS()`. That field remains explicitly unverified in the CSV.

Original scene SHA-256: 8a40f6a1a5c47eacbbe92920e131f4cae04cfb0de6be1991b4719ea2860bd2c.

Loaded Scatter DLL SHA-256: c3656015ff349e0e9a8382eb694cc24ce39620da6d0758424abcb9d2750609fc.

Installed/repository script SHA-256: 34062594ce2006833ba3f281eae8fabcd4d2184c95955c9dfc092c7a597376391.

The existing [performance monitor](#) was loaded headlessly and used for serialization, state observations and independent process sampling. The new [navigation harness](#) measures redraw work separately from rebuild operations. The [summarizer](#) derives median, nearest-rank P95, repeat medians and counter deltas from raw CSV/JSON.

Scene population

Layer	Generated	Displayed proxies	Preview triangles
Grass	44,958	2,000	12,000
Leaves	200	200	1,200
clover	3,529	2,000	12,000
BushesCenter	70	40	240
Bourder	3,342	2,000	12,000
Street_Plant	44	44	264
Total	52,143	6,284	37,704

Root settings: Proxy mode, Pyramid shape, 2,000 instances/layer, 2,000,000 faces/layer, point budget 500,000, 1,000 samples/source, all six layers enabled. Radius overlays are empty. The Analyzer displays boundary, paths, street lines and nine point markers.

Three Corona .cgeo files referenced on an unavailable E: path are missing: Lavander_01.cgeo, Lavander_05.cgeo and MT_PM_V33_Glossy_abelia_01_03.cgeo. Their native geometry-preview eligibility tests return no displayed geometry. This accounts for the BushesCenter difference between 70 placements and 40 displayed proxies. Restore those assets before renderer qualification. No replacement geometry or path repair was silently introduced into the baseline.

Point Cloud mode has a separate, more severe result in this scene: source sampling raises Cannot sample source geometry and clears the **entire BushesCenter layer preview**, including its valid mesh sources. All three point-cloud budgets reproduce this error; their timing results describe an incomplete display. The main pyramid baseline and accepted geometry prototypes do not contain this sampling error.

The viewport's scene polygon statistics report approximately 1.83 million polygons. That is a different quantity from the 37,704 triangles submitted by the custom scatter overlay. Use the native preview face counter to describe the overlay workload.

Protocol and limitations

- Main matrix: 3 repeats x 60 measured steps, 8 warmups each; the second repeat reverses most condition ordering. Only camera orientation changes during measured movement, over a repeatable +/-6-degree path.
- Display matrix: 3 x 40 measured steps per setting, with explicit cache rebuilds before measuring. Display reduction is a quality/performance tradeoff, not equal-workload acceleration.
- Drawing prototypes: 3 x 60 steps per case; exact per-layer displayed counts and triangle totals checked before accepting results.
- Layer profiling: 60 steps plus warmups; a diagnostic callback mirrors production ordering and records inclusive native draw time per layer.
- Additional controls: stationary redraw, maximized viewport, Analyzer callback without `gw.updateScreen()`, and actual mouse pan gestures.
- Total retained main/control measurements: 3,900 steps. Of these, 3,540 have no recorded preview error; 360 point-cloud steps retain the known BushesCenter sampling failure and are classified as degraded-display observations.
- GPU completion, GPU busy time, presentation latency and exact CPU-instruction attribution were not measured. Native callback wall time can include driver work or synchronization.
- Desktop activity and message dispatch produce tails. P95 describes these recordings, not a universal service-level guarantee. Repeat-level medians remain in the evidence.

The step normally contains two callback executions. A phase probe found one in `redrawViews()` and approximately one more during message processing; `viewport.setTM` itself made none in that probe. Maximizing the viewport did not halve the time. **Do not attribute this factor of two to two visible viewports or divide the measured times by two and call the result FPS.**

3. Root cause and code path

Why Manual remains slow

Manual mode controls whether a dirty layout is rebuilt. It does not suppress drawing of the existing preview cache. The relevant sequence is:

1. Max requests a viewport redraw.
2. `AminScatterObjectDraw` finds visible Scatter controllers, builds icon paths and calls `layerEntries()`.
3. Each layer synchronizes controller-owned settings, checks revisions, calls `previewCache()` twice and calls `drawRadii()`.
4. `aminScatterDrawPreview` walks every cached source and selected instance.
5. For each instance it changes the `GraphicsWindow` transform, begins triangles, calculates face shading and submits individual triangles, then ends triangles.

Source locations:

Location	Verified behavior	Consequence
<code>AminScatter/src/preview.cpp</code> :114-121	Nested source/instance loop; <code>setTransform/startTriangles/endTriangles</code> per instance	6,284 tiny triangle batches per callback for this scene
<code>preview.cpp</code> :117-119	Transform two edges, cross/normalize, light dot product and per-vertex colors for every face	Repeated CPU-side preparation on unchanged geometry
<code>geometry_preview.inc</code> :37-43	Evenly select requested rows, enforce instance/face caps, retain source faces and transforms	A CPU cache exists; the draw submission itself is not retained
<code>AminScatterObject.ms</code> :11535-11550	Whole-scene traversal, icon generation, layer traversal, two <code>previewCache</code> calls, whole-window invalidation	Several milliseconds of script work remain even with cheap drawing
<code>AminScatterObject.ms</code> :11265-11297	Surface/settings synchronization and edit/revision checks in <code>layerEntries</code>	Camera redraw pays control-plane work unrelated to changed placement
<code>CyrusSurfaceAnalyzer.ms</code> :371-392	Reconstruct/display line data and markers; enlarge whole update rectangle; <code>updateScreen</code>	Secondary overlay cost; explicit <code>updateScreen</code> is not the main defect in this test

The native cache is genuinely reused: this investigation found no camera-induced placement rebuild storm in the measured scene. The previous CPU optimization remains useful for edits and explicit refreshes, but it cannot remove per-redraw submission cost.

Per-layer attribution

Median native drawing time **per callback**, measured with the same displayed population:

Layer	Native draw
Grass	9.68 ms
Leaves	1.00 ms
clover	9.44 ms
BushesCenter	0.24 ms
Bourder	9.30 ms
Street_Plant	0.26 ms

The three 2,000-instance layers dominate. At controller level, median icon generation/drawing was about 0.44 ms and `LayerEntries()` about 2.19 ms per callback. Each of the two cache checks per layer cost roughly 0.18-0.33 ms; empty radius calls were around 0.02 ms. These separately measured medians are attribution clues, not values to sum into an exact frame total.

The controlled original case spends a median 73.63 ms per step inside the Scatter callback and 2.44 ms inside the Analyzer callback. Four direct mouse pan strokes corroborated the same path without automated camera updates: the two captured mouse-held Scatter callbacks had a median 48.87 ms, with zero rebuilds. That small gesture sample is corroboration only, not a separate FPS benchmark.

4. Experiments that establish the improvement path

Matched-count geometry experiments

All prototype cases represent the same 6,284 displayed pyramids and 37,704 triangles. The disposable meshes use the same placement routine and row-selection rule, source-local bounding pyramids and final placement transforms. Eligibility is checked with the production native preview builder, so missing proxies remain skipped.

Representation	Median step	P95 step	What changes
Original callbacks	90.52 ms	142.40 ms	One triangle batch per displayed proxy
World-space batches, original script plumbing	31.56 ms	43.75 ms	Six batches; same traversal/cache checks/icons/radius calls
World-space batches, minimal callback, Analyzer on	17.93 ms	28.92 ms	Removes redundant script traversal from the experiment
World-space batches, minimal callback, Analyzer off	13.90 ms	20.32 ms	Also removes Analyzer overlay
Six ordinary Max meshes, Analyzer on	11.13 ms	18.71 ms	Max handles geometry display; Scatter callback absent
Six ordinary Max meshes, Analyzer off	6.69 ms	12.02 ms	Remaining scene plus ordinary mesh display

The world-space version still calls the **existing native draw primitive**, including its per-face shading loop. The major speedup therefore does not require a faster scatter algorithm or GPU placement compute. Grouping geometry and avoiding per-instance GraphicsWindow setup is a demonstrated high-value direction.

These are **diagnostic prototypes**, not production-ready replacements. Source colors are simplified into layer colors, mesh shading differs, source bounding extraction is a script reconstruction, and no pixel-perfect or byte-for-byte

geometry parity was certified. The tests establish count-preserving performance potential; they do not establish a release speedup. Ordinary meshes also remove the original script plumbing and benefit from Max's normal geometry handling, so their full gain must not be attributed to batching alone.

The matched meshes took approximately 0.8 seconds to construct through MAXScript outside timed navigation. That construction includes evaluating placements and is not a prediction for a native cache builder. All temporary nodes were removed; no production node baking was introduced.

Display-budget choices available today

Setting	Displayed items	Preview triangles	Median step
Pyramid, 2,000/layer	6,284 proxies	37,704	89.10 ms
Pyramid, 1,000/layer	3,284 proxies	19,704	59.61 ms
Pyramid, 500/layer	1,784 proxies	10,704	43.44 ms
Pyramid, 250/layer	1,034 proxies	6,204	34.84 ms
Pyramid, 100/layer	484 proxies	2,904	28.90 ms
Box, 2,000/layer	6,284 proxies	75,408	93.53 ms
Point Cloud, 500,000 budget; BushesCenter error	357,506 points	0	54.89 ms
Point Cloud, 50,000 budget; BushesCenter error	40,304 points	0	26.89 ms
Point Cloud, 10,000 budget; BushesCenter error	8,281 points	0	23.85 ms
All centers	52,143 points	0	28.81 ms

For immediate navigation, use **250-500 pyramid proxies per layer** when volume is useful, or **all centers** for a light representation that includes all 52,143 placements. Restore the preferred display quality after navigation. The 10,000-50,000 Point Cloud settings show lower wall time but omit BushesCenter; only recommend them after source sampling and layer visibility are verified. Manual mode controls rebuilds during edits but is not a navigation optimization by itself.

Point budget is divided between enabled layers and then reduced by stride selection; displayed counts need not equal the entered budget. `showCenters` uses a separate 500,000-point ceiling in the current implementation. Instance and face limits are per layer, so several layers multiply the controller's total burden. The same 6,284 boxes doubled triangle count while adding only about 4.4 ms over pyramids, consistent with substantial per-instance overhead; it does not independently quantify every native cost.

Negative controls

- Maximized preview: 90.36 ms versus 90.04 ms in the two-view layout. Maximizing is not an effective remedy here, though it changes pixel area and framing conditions.
- Stationary forced redraw: 92.78 ms. The cost is incurred even when the camera is not changing.
- Remove only Analyzer `gw.updateScreen()`: 90.94 ms versus 91.86 ms in the paired original control. This small difference does not justify treating `updateScreen` as the central fix.
- Resource sampler: 132 samples, no sampler error, sampled peak private memory about 3.86 GiB and working set about 2.33 GiB during the main matrix. These readings do not suggest RAM capacity pressure as the measured cause; they do not certify absence of leaks or GPU-memory pressure.

5. Prioritized implementation guideline

P0 - Batch the existing proxy drawing path

Primary files: AminScatter/src/preview.cpp, geometry_preview.inc; budget/status settings in generator inputs and regenerated AminScatterObject.ms.

Add immutable, bounded proxy draw batches to the preview cache. Build them when selected placements, source bounds or display shape change. Emit triangles by layer/source/color chunk with identity/world transform rather than changing GraphicsWindow state for each tiny proxy. Retain the original ordered selected rows and source IDs. Point placeholders must continue to use markers; missing or non-convertible sources must preserve the existing skip behavior.

For this scene, 37,704 triangles require about **1.29 MiB for raw float XYZ positions alone** at 36 bytes/triangle. This is practical for a proxy experiment. It is not permission to expand every full source mesh: 20 million triangles require about 687 MiB for positions alone, before colors, indices, buffers and allocator overhead.

Use an explicit memory budget and bounded chunks. Start with proxy modes; keep the current path as a measured fallback for oversized caches during development. Define per-controller and scene-wide accounting before shipping. Do not silently alter render count, seed, layer composition or placement selection to make the test pass.

Precompute stable shading factors at cache construction where beneficial. A scalar shade can remain separate from source/solid color so color-only changes avoid rebuilding positions. Handle mirrored/nonuniform transforms using the same geometric normal semantics as the existing cross-product path. Hoisting the constant light direction and eliminating repeated normalization are small supporting changes; measure their effect rather than promising a particular gain.

Initial gate: reproduce the saved scene at the original 6,284/37,704 population, zero navigation rebuilds, median step ≤ 35 ms and P95 ≤ 55 ms under this exact harness on this machine. These are proposed engineering targets informed by the prototype, not achieved product metrics. Repeat with real navigation before publishing FPS claims.

P1 - Make redraw consume prepared display state

Primary files: AminScatter/tools/ui/templates/before.ms, edit.cjs, performance.cjs, radius-display.cjs, relevant display generator stages and CyrusSurfaceAnalyzer/scripts/CyrusSurfaceAnalyzer.ms.

Move dependency synchronization and dirty-resolution out of the hot drawing traversal where safe. Maintain a controller/layer draw list with explicit membership and revision invalidation. Query interaction state once per redraw, retrieve each preview cache once, and avoid repeated refs.dependentNodes, settings comparisons and temporary arrays when nothing changed. Cache icon vertices and Analyzer packed line lists until their underlying data changes.

The intended contract is: **drawing consumes a ready snapshot; editing prepares the next snapshot**. Initially preserve current synchronous behavior outside active drags and the first-build fallback so a fresh/opened scene never silently loses its preview. Do not just remove layerEntries() or revision checks: surface changes, Analyzer output, overlap blockers, edit-stack state, disabled layers, load/clone/Undo and time changes all affect cache validity.

Controller registration must handle create/delete, hide/unhide, layer changes, clone, load/reset and invalid nodes. Source geometry must be keyed by actual evaluated geometry/time/offset transforms, not just node handles. Cache versioning should distinguish placements, representation geometry, color/style and view-dependent selection.

Evaluate whole-window invalidation and explicit screen updates after the dominant batch fix. The Analyzer no-flush trial is a negative result for prioritization, not proof that the existing redraw policy is optimal in every context.

Gate: after P0, reduce the empty/unchanged script traversal contribution, preserve all invalidation cases, and prove that camera changes do not increment preview, Analyzer or PFlow counters. Include multiple controllers and disabled layers; this scene has only one controller.

P2 - Retained geometry and viewport instancing

Use a dedicated design spike for the Max display integration. Autodesk's `IObjectDisplay2/render-item` path and `ViewportInstancing::InstanceDisplayGeometry` provide mechanisms for retaining source geometry and updating instance data. The installed Max 2027 SDK includes these interfaces. GPU viewport instancing addresses drawing; OpenCL/CUDA placement generation addresses a different workload.

For full Mesh mode, prefer one source geometry buffer plus instance transforms over expansion into every instance's world triangles. Point Cloud can use retained point buffers and explicit color groups. Introduce per-view bounds, frustum rejection and a controlled screen-size or interaction LOD policy once the basic representation is correct.

The existing owner is a scripted `SimpleObject`. A retained display adapter is **not a drop-in change to** `AminScatterDrawPreview`. Prove how a compiled display component attaches to the existing controller without breaking class IDs, references, saving or picking. Compare a native display adapter with an owned transient display component; document lifecycle and scene-explorer implications before choosing. Avoid a broad persistent-class migration as an incidental performance patch.

SDK integration details to verify: world-space versus owner-relative transforms, `CreateInstanceData` versus `UpdateInstanceData` restrictions, source/topology count changes, vertex/material streams in Standard and High Quality views, `optimesh.lib`, graphics resource lifetime and device reset. Keep Max SDK object evaluation and scene mutation on their supported threads; only independent numeric preparation belongs in worker jobs.

Gate: same visible population, stable bounds and picking, source/solid color parity, hidden/frozen/viewport-filter behavior, wireframe/edged/shaded views, multiple viewport configurations, device reset and repeated file open/reset. A future ≤ 16.7 ms navigation-step target is a design objective, not a certified 60-FPS result.

P3 - Product budgets and diagnostics

Expose generated placements, displayed instances, submitted triangles/points, draw-batch count and estimated CPU/GPU cache bytes separately. Display a clear reason when sources are skipped or a budget is reached. Provide navigation-quality presets and a scene-wide cap so ten layers or several controllers cannot multiply a benign per-layer setting into an unexpectedly large workload.

Retain manual artist overrides. During interaction, choose a stable subset; avoid random reshuffling or visible flicker as the camera moves. Quality reduction must affect preview only. Keep render/PFlow output governed by the original placement settings.

The missing Corona proxies are a diagnostics issue as well as a test limitation. Report them explicitly; a displayed-count shortfall should not be mistaken for a computation failure or an overlap rule.

Repair the all-or-nothing Point Cloud failure policy deliberately. `max_bridge.cpp:378` raises the sampling error, while `AminScatterObject.ms:11201-11205` samples every source inside one layer-wide try/catch and clears the cache on any failure. Preserve source-index mapping when introducing per-source skip or placeholder behavior, expose the skipped source and reason, and test mixed valid/missing sources. Do not silently replace missing geometry in render output. This is a correctness/diagnostics improvement as well as a usability fix.

P4 - Continue the separate editing-performance track

The October 1 CPU work reduced explicit calculation time, but earlier artist traces still show repeated dependent rebuild passes. Continue investigating surface -> Analyzer publication -> blocker/source revisions -> dependent scatter ordering. Reuse evaluation-owned data and only suppress a pass when actual input/output revisions prove it redundant.

Do not start GPU placement compute, broader threading, sampler changes or render/PFlow rewrites to solve this particular idle-navigation defect. They remain separate opportunities, with independent profiling and exact placement-output regression gates.

6. Codebase-wide performance map

This review followed the complete performance path across native generation, bridge, generated controller, display, Analyzer, CS Edit, renderer transport, installation and test tooling. The table distinguishes fresh navigation measurements from source concerns and previous evidence. It is not a claim that every feature has been runtime-qualified.

Subsystem	Review outcome	Appropriate next action
Native scatter, cluster, prepared-band and execution modules	Not entered for measured steady navigation; current CPU optimizations already exist	Preserve RNG order, ordered acceptance and serial/parallel parity; profile edits separately
Boundary/orientation/falloff/final spacing	Geometry search and repeated predicates matter during evaluation, not this unchanged redraw	Reuse prepared boundary/surface structures only behind exact-output checks
Native Max bridge	Evaluates meshes and packs/unpacks MAXScript data during preparation	Cache evaluation-owned source data; no SDK calls from unsupported worker threads
Point and geometry preview cache builders	Source samples/geometry built during refresh; only final draw path dominates here	Separate immutable draw snapshot from placement generation and budget selection
Generated controller and UI stages	Per-redraw synchronization and duplicate cache access measured	Edit generator sources and regenerate; do not hand-patch generated production output
Surface Analyzer core and overlay	No measured analysis runs; drawing has smaller steady cost	Cache line arrays/icons; profile analysis and publication order during edits
CS Edit storage/stack/display	Absent from this scene; <code>visible()</code> reconstructs positions for display/bounds/hit-test	Separate visible-position cache with full activity/input/Undo invalidation, not <code>editRevision</code> alone
PFlow/Corona integration	No PFlow builds during navigation; IR and production render not tested	Preserve transport ownership, grouping and save/reset cleanup; qualify separately
Install/build/version paths	Installed candidate matches the identified native/script files	Keep host-year ABI and generated-source checks; test other Max versions independently
Monitoring/tests	Existing monitor is rebuild-oriented; new harness records navigation and counter invariants	Keep wall-time, completed-frame, rebuild and input-to-preview metrics distinct

Existing background timers are not proof of background computation. Mouse-release polling, event coalescing and Analyzer/IR timers serve different purposes. No measured navigation rebuilds means turning those mechanisms off is not the first corrective action here.

7. Validation and release gates

1. **Freeze evidence:** scene hash, native/script hashes, driver, viewport settings/dimensions, generated/displayed/triangle counts, color mode, missing assets and callback configuration.
2. **Change one layer of the system:** proxy batching first, script traversal second, retained backend later. Re-run the same saved scene with the same visible population after each change.
3. **Measure correctly:** three or more repeats, warmups, median and P95, repeat medians and zero measured rebuild counters. Capture real drag/orbit latency and an appropriate presented-frame trace before making FPS claims.
4. **Check output:** ordered transforms/source assignment from existing differential harnesses, deterministic display-row selection, exact proxy topology and bounds, floating-point tolerances documented where necessary. Screenshot differences must be explained and accepted, not concealed.
5. **Check difficult inputs:** zero placements, missing/deleted sources, negative/nonuniform scales, large coordinates, animated source geometry, point placeholders, all proxy shapes, Mesh mode, face/instance/point limits, many sources/layers/controllers and disabled layers.
6. **Check lifecycle:** open/reset/load/clone, save/reopen, hide/filter/isolate/freeze, selection and CS Edit picking, Undo/Redo, display-mode switches and repeated memory/device-resource release.
7. **Check integration:** actual Modify-panel editing, Analyzer-dependent changes, Corona IR/production/save cleanup and each supported Max year. This investigation qualifies neither render output nor other host years.

Run existing native/Max regression suites for production changes to shared geometry or placement code. A viewport-only optimization must not silently change rendering, serialization, scene ownership or generation semantics. Record both the faster drawing result and any added rebuild/memory cost.

8. Reproduction tools and evidence register

The durable [evidence package](#) contains raw frame/case files, summaries, per-layer stages, frame phases, mouse-drag observations, settings, module identities, process sampling and restoration records. Its validation distinguishes 3,540 error-free timing steps from 360 degraded point-cloud steps. Investigation recipes are retained alongside it. The 185 MB scene copy is intentionally kept in `build/viewport-investigation-2026-10-01`, not duplicated into documentation.

To inspect a recorded run:

```
python tools/performance/summarize_viewport.py build/viewport-investigation-2026-10-01/matrix
```

For a new scene, use a saved copy. Load the existing monitor with `CyrusPerfHeadless=true`, then load `CyrusViewportBenchmark.ms`. Set `CVBControllers`, `CVBAnalyzers` and `CVBBaseTM` from the actual scene, use `CVBCallbacks()` to enable wrappers, run `CVBRunCase`, write results with `CVBWrite`, then always call `CVBRestoreCallbacks()` and restore the camera. Warm the preview first. The supplied recipes show the exact case preparation used here; they are investigation fixtures, not a general unattended scene editor.

A callback-disabled case is an isolation control, not a usable optimized scatter. A reduced-count case is a quality tradeoff. A count-preserving prototype demonstrates feasibility, not production parity. Keep these categories separate in future reports.

Initial harness-development failures were corrected before accepting data: inventory formatting encountered an empty source slot, and the first ordinary-mesh prototype included three sources that the production cache omits. The accepted prototype checks native source eligibility and asserts every layer's displayed instance and triangle totals. Failed attempts are retained in the working investigation folder and are not included as successful trials.

The original scene was not saved over; installed plugin files were not replaced. Temporary diagnostic scene nodes and callbacks were removed, original drawing callbacks and viewport settings restored, and the local command timers disposed. The original scene was reopened for continued use. The repository already contained extensive uncommitted CPU-performance work; this investigation adds measurement tooling and documentation without replacing that work.

9. Sources and related work

- [Autodesk: viewport transforms and GetFPS](#) - camera controls and the Statistics/adaptive-degradation limitations of reported FPS.
- [Autodesk: refreshing viewports](#) - ordinary redraw versus forced complete regeneration; this harness uses `redrawViews`.
- [Autodesk: plugin display interface](#) - retained display/render-item integration.
- [Autodesk: InstanceDisplayGeometry](#) and [header reference](#) - viewport GPU instance data and update contract. The installed matching SDK header was also inspected.
- [Measured CPU implementation](#), [artist editing retest](#) and [existing viewport plan](#) - prior calculation/editing evidence and design context. This investigation supplies the previously missing navigation measurements and raises batching/retained display above shade-only optimization.

Recommended next engineering task: implement bounded native proxy batching with unchanged displayed population and explicit cache invalidation, then validate the ≤ 35 ms navigation-step target. Follow with removal of repeated script work from the redraw path. Keep retained GPU instancing as the scalable display architecture, particularly for Mesh mode.